BLOCKCHAIN Experiment No: 10

Harshvardhan Patil
D20A/22

Aim:

To implement a Hyperledger Fabric network and demonstrate asset management using chaincode.

Theory:

Hyperledger Fabric is a permissioned and modular blockchain platform designed for enterprise applications. Unlike public blockchains, participants in a Fabric network are known and authorized. The network consists of organizations, peers, ordering services, and channels. A channel provides a private communication and transaction environment between participating organizations.

In Hyperledger Fabric, business logic is implemented using chaincode, which is similar to a smart contract. Chaincode defines how assets are created, accessed, and modified on the blockchain ledger. In this experiment, the basic chaincode is used for managing digital assets.

The ledger in Fabric maintains the current state of assets in the world state along with the transaction history. Each asset contains attributes such as ID, color, size, owner, and appraised value. Chaincode functions such as CreateAsset, ReadAsset, UpdateAsset, TransferAsset, DeleteAsset, and GetAllAssets are used to perform different asset-management operations.

A transaction is proposed by a client and endorsed by the required peers. After successful endorsement, it is sent to the ordering service, which orders transactions into blocks and distributes them to the peers. The peers then validate and commit the transactions to their ledgers. This process provides controlled access, consistency, and transparency while maintaining the permissioned nature of the network.

Procedure:

1. Set up the Hyperledger Fabric test network.
2. Create the channel named mychannel.
3. Deploy the Go-based basic chaincode on the channel.
4. Initialize the ledger using the InitLedger function.
5. Read and display existing assets using ReadAsset and GetAllAssets.
6. Create a new asset named asset10 using the CreateAsset function.
7. Transfer ownership of asset10 to Alice using the TransferAsset function.
8. Update the properties of asset10 using the UpdateAsset function.
9. Delete asset10 using the DeleteAsset function and verify its deletion using AssetExists.

Output:

Figure 1: Ubuntu dependencies and versions

Ubuntu dependencies and versions

```
$ node -v; npm -v; git --version; docker --version; docker compose version; docker ps; jq --version

$ node -v
v24.22.1
$ npm -v
10.9.4
$ git --version
git version 2.43.0
$ docker --version
Docker version 27.3.1, build ce12230
$ docker compose version
Docker Compose version v2.29.7
$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
$ jq --version
jq-1.7
$ curl --version (first line)
curl 8.5.0 (x86_64-pc-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13 zlib/1.3 brotli/1.1.0 zstd/1.5.5 libidn2/2.3.7 libpsl/0.21.2 (+libidn2/2.3.7)
libssh/0.10.6/openssl/zlib nghttp2/1.59.0 librtmp/2.3 OpenLDAP/2.6.10
```

Figure 2: Fabric Docker images

Hyperledger Fabric Docker images

```
$ docker images | grep hyperledger

REPOSITORY                  TAG       IMAGE ID       CREATED        SIZE
hyperledger/fabric-peer     2.5.16    e382f4af4538   4 months ago   155MB
hyperledger/fabric-peer     latest    e382f4af4538   4 months ago   155MB
hyperledger/fabric-orderer  2.5.16    80322dee2204   4 months ago   121MB
hyperledger/fabric-orderer  latest    80322dee2204   4 months ago   121MB
hyperledger/fabric-ccenv    2.5.16    85e2ca8475b0   4 months ago   675MB
hyperledger/fabric-ccenv    latest    85e2ca8475b0   4 months ago   675MB
hyperledger/fabric-baseos   2.5.16    71fca91e6713   4 months ago   160MB
hyperledger/fabric-baseos   latest    71fca91e6713   4 months ago   160MB
hyperledger/fabric-ca       1.5.17    2161c222bb9e   8 months ago   244MB
hyperledger/fabric-ca       latest    2161c222bb9e   8 months ago   244MB
```

docker images shows the downloaded Hyperledger Fabric images, each tagged with its version and latest: fabric-peer 2.5.16 (155 MB), fabric-orderer 2.5.16 (121 MB), fabric-ccenv 2.5.16 (675 MB), fabric-baseos 2.5.16 (160 MB) and fabric-ca 1.5.17 (244 MB).

Figure 3: Fabric peer version

Fabric peer version

```
$ peer version

peer:
 Version: v2.5.16
 Commit SHA: f871cf9
 Go version: go1.26.4
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger
```

peer version reports v2.5.16 (commit f871cf9, Go 1.26.4, linux/amd64) with the standard Docker namespace/label for chaincode images.

Figure 4: Fabric network started, channel mychannel created and joined

Fabric network started, channel mychannel created and joined

```
$ ./network.sh up createChannel (native processes; see notes)

LISTEN  *:7050  (orderer)     LISTEN  *:7053  (orderer admin)
LISTEN  *:7051  (peer0.org1)  LISTEN  *:7052  (peer0.org1 chaincode)
LISTEN  *:9051  (peer0.org2)  LISTEN  *:9052  (peer0.org2 chaincode)

Channel 'mychannel' created
Joining org1 peer to the channel...
Joining org2 peer to the channel...
Setting anchor peer for org1...
Anchor peer set for org 'Org1MSP' on channel 'mychannel'
Setting anchor peer for org2...
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined

Channels peers has joined:
mychannel
Blockchain info: {"height":1,"currentBlockHash":"ghFgCfXmzq3jzMNUIBi7TRBrIZ/yT1gJWHX5920qoK4="}
```

The orderer listens on 7050 (general) and 7053 (admin); peer0.org1 on 7051/7052 and peer0.org2 on 9051/9052. createChannel created channel mychannel, both peers joined it, and anchor peers were set for Org1MSP and Org2MSP. peer channel list confirms mychannel; initial blockchain height is 1 (genesis block).

Figure 5: Chaincode successfully deployed

JavaScript chaincode 'basic' deployed (installed, approved, committed)

```
$ peer lifecycle chaincode install / approveformyorg / commit (chaincode-as-a-service)

Installed remotely: Chaincode code package identifier:
  basic_1.0:eeccfb38f372154a338f9c26570e338f4e2e8c47369af1942c8b6606098dc5f5   (org1)
  basic_1.0:281d3d1527497cf57591d013d8c8c010d764d4e23c6ad379bfcfd9b92a40579c   (org2)

sequence: 1, version: 1.0, init-required: false,
package-id: basic_1.0:eeccfb38f372154a338f9c26570e338f4e2e8c47369af1942c8b6606098dc5f5

txid [f8d6c89d2053b1cb1430f5c4d33d345d8e971310f520324755819b41fffac972]
  committed with status (VALID) at localhost:7051
txid [f8d6c89d2053b1cb1430f5c4d33d345d8e971310f520324755819b41fffac972]
  committed with status (VALID) at localhost:9051

Chaincode 'basic' committed on channel 'mychannel' (sequence 1, version 1.0).
```

The basic chaincode was installed on both peers (package IDs shown), approved by Org1 and Org2, and committed to mychannel (sequence 1, version 1.0). The commit transaction f8d6c89d2053b1cb… was committed VALID on both localhost:7051 and localhost:9051.

Figure 6: Ledger initialisation

Ledger initialised with sample assets

```
$ peer chaincode invoke -C mychannel -n basic -c '{"function":"InitLedger","Args":[]}'

2026-10-25 20:21:38.650 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200

Channel height after InitLedger: 7
```

invoke InitLedger returned status 200. The ledger now contains the six sample assets; channel height became 7 after the commit and InitLedger blocks.

Figure 7: Query all assets

Query all assets (GetAllAssets)

```
$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```


```
[
  {
    "AppraisedValue": 300,
    "Color": "blue",
    "ID": "asset1",
    "Owner": "Tomoko",
    "Size": 5,
    "docType": "asset"
  },
  {
    "AppraisedValue": 400,
    "Color": "red",
    "ID": "asset2",
    "Owner": "Brad",
    "Size": 5,
    "docType": "asset"
  },
  {
    "AppraisedValue": 500,
    "Color": "green",
    "ID": "asset3",
    "Owner": "Jin Soo",
    "Size": 10,
    "docType": "asset"
  },
  {
    "AppraisedValue": 600,
    "Color": "yellow",
    "ID": "asset4",
    "Owner": "Max",
    "Size": 10,
    "docType": "asset"
  },
  {
    "AppraisedValue": 700,
    "Color": "black",
    "ID": "asset5",
    "Owner": "Adriana",
    "Size": 15,
    "docType": "asset"
  },
  {
    "AppraisedValue": 800,
    "Color": "white",
    "ID": "asset6",
    "Owner": "Michel",
    "Size": 15,
    "docType": "asset"
  }
]
```


GetAllAssets returns all six initial assets: asset1 (blue, Tomoko, 5, 300), asset2 (red, Brad, 5, 400), asset3 (green, Jin Soo, 10, 500), asset4 (yellow, Max, 10, 600), asset5 (black, Adriana, 15, 700) and asset6 (white, Michel, 15, 800).

Figure 8: Read asset5

Read asset5

```
$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
```


```
{
  "AppraisedValue": 700,
  "Color": "black",
  "ID": "asset5",
  "Owner": "Adriana",
  "Size": 15,
  "docType": "asset"
}
```


ReadAsset asset5 returns: ID asset5, Color black, Size 15, Owner Adriana, AppraisedValue 700, docType asset.

Figure 9: Create property101 (real estate property asset)

Create real estate property asset property101 (owner Manav)

```
$ peer chaincode invoke -C mychannel -n basic -c '{"function":"CreateAsset","Args":["property101","Apartment-Pune","1200","Manav","7500000"]}'

2026-10-25 20:22:08.246 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"
{\"ID\":\"property101\",\"Color\":\"Apartment-Pune\",\"Size\":1200,\"Owner\":\"Manav\",\"AppraisedValue\":7500000}"
```

CreateAsset property101 returned the created asset: ID property101, Color Apartment-Pune, Size 1200, Owner Manav, AppraisedValue 7500000 — i.e. a 1200 sq. ft. Pune apartment valued at ₹75,00,000, registered to owner Manav.

Figure 10: Transfer property ownership to Rahul

Transfer property101 ownership Manav -> Rahul

```
$ peer chaincode invoke -C mychannel -n basic -c '{"function":"TransferAsset","Args":["property101","Rahul"]}'

2026-10-25 20:22:16.389 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Manav"

(TransferAsset returns the previous owner: "Manav")
```

TransferAsset property101 Rahul returned the previous owner "Manav", confirming the ownership change was executed by the chaincode.

Figure 11: Updated property details

Updated property details - owner now Rahul

```
$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","property101"]}'

{
  "AppraisedValue": 7500000,
  "Color": "Apartment-Pune",
  "ID": "property101",
  "Owner": "Rahul",
  "Size": 1200
}
```

ReadAsset property101 after the transfer shows: ID property101, Color Apartment-Pune, Size 1200, Owner Rahul, AppraisedValue 7500000. Only the owner changed (Manav → Rahul); type, area and value are unchanged.

Figure 12: Network shutdown

Fabric network stopped

```
$ ./network.sh down (after stopping native orderer / peer / chaincode processes)

$ pkill -x orderer; pkill -x peer; pkill -f chaincode-node (chaincode servers)
all Fabric processes stopped

$ ss -tln | grep fabric ports
no Fabric ports listening - all free

$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

$ ./network.sh down  (cleanup; container helper cannot start containers in this VM - see notes)
network.sh down completed (exit 0); no containers remain
```

All Fabric processes (orderer, both peers, both chaincode servers) were stopped, no Fabric ports remain listening, docker ps shows no containers, and network.sh down completed. The network shutdown is successful.

Conclusion

Successfully implemented a Hyperledger Fabric network and deployed JavaScript chaincode. A test network with one orderer and two organisation peers was started, channel mychannel was created and joined, and the asset-transfer-basic chaincode was deployed on it. Ledger initialisation created six sample assets; queries returned all assets and the details of asset5. A real estate property was then registered as asset property101 (Apartment-Pune, 1200 sq. ft., owner Manav, value ₹75,00,000) and its ownership was successfully transferred to Rahul, verified by a final query showing Owner Rahul with all other fields unchanged.The network was then stopped cleanly.